

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I ADHAVAN CHANDRASEKAR(192521209) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ADHAVAN CHANDRASEKAR(192521209)

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

	correct final answer	<pre> 1 -- Create Department Table 2 CREATE TABLE DEPARTMENT 3 (4 DeptID INT PRIMARY KEY, 5 DeptName VARCHAR(20) 6); 7 8 -- Create Student Table 9 CREATE TABLE STUDENT 10 (11 RegNo INT PRIMARY KEY, 12 Name VARCHAR(30), 13 DeptID INT, 14 FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID) 15); 16 17 -- Insert Sample Data 18 INSERT INTO DEPARTMENT VALUES 19 (1, 'CSE'), 20 (2, 'ECE'), 21 (3, 'MECH'); 22 23 INSERT INTO STUDENT VALUES 24 (101, 'Arun', 1), 25 (102, 'Priya', 2), 26 (103, 'Rahul', 1), 27 (104, 'Sneha', 3); </pre>		
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

ANSWERS

- Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

```

1  -- Create Department Table
2  CREATE TABLE DEPARTMENT
3  (
4      DeptID INT PRIMARY KEY,
5      DeptName VARCHAR(20)
6  );
7
8  -- Create Student Table
9  CREATE TABLE STUDENT
10 (
11     RegNo INT PRIMARY KEY,
12     Name VARCHAR(30),
13     DeptID INT,
14     FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
15 );
16
17 -- Insert Sample Data
18 INSERT INTO DEPARTMENT VALUES
19 (1, 'CSE'),
20 (2, 'ECE'),
21 (3, 'MECH');
22
23 INSERT INTO STUDENT VALUES
24 (101, 'Arun', 1),
25 (102, 'Priya', 2),
26 (103, 'Rahul', 1),
27 (104, 'Sneha', 3);
28

```

QUERY 1 : Inner Join

```
30  SELECT S.RegNo, S.Name, D.DeptName
31  FROM STUDENT S
32  INNER JOIN DEPARTMENT D
33  ON S.DeptID = D.DeptID;
```

OUTPUT:

RegNo	Name	DeptName
101	Arun	CSE
103	Rahul	CSE
102	Priya	ECE
104	Sneha	MECH

QUERY 2 : Left Join

```
36  SELECT S.RegNo, S.Name, D.DeptName
37  FROM STUDENT S
38  LEFT JOIN DEPARTMENT D
39  ON S.DeptID = D.DeptID;
40
```

OUTPUT :

RegNo	Name	DeptName
101	Arun	CSE
102	Priya	ECE
103	Rahul	CSE
104	Sneha	MECH

QUERY 3 : Subquery - Students in CSE

```

42  SELECT Name
43  FROM STUDENT
44  WHERE DeptID = (
45      SELECT DeptID
46      FROM DEPARTMENT
47      WHERE DeptName = 'CSE'
48  );
49

```

OUTPUT:

```

+-----+
| Name  |
+-----+
| Arun  |
| Rahul |
+-----+

```

QUERY 4 : Subquery - Students in ECE

```

51  SELECT Name
52  FROM STUDENT
53  WHERE DeptID = (
54      SELECT DeptID
55      FROM DEPARTMENT
56      WHERE DeptName = 'ECE'
57  );
58

```

OUTPUT:

```

+-----+
| Name  |
+-----+
| Priya |
+-----+

```

QUERY 5 : Count Students in Each Department

```

60  SELECT D.DeptName, COUNT(S.RegNo) AS TotalStudents
61  FROM DEPARTMENT D
62  INNER JOIN STUDENT S
63  ON D.DeptID = S.DeptID
64  GROUP BY D.DeptName;

```

OUTPUT :

```

| DeptName | TotalStudents |
+-----+-----+
| CSE      | 2             |
| ECE      | 1             |
| MECH     | 1             |
+-----+-----+

```

- 2. Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.**

STUDENT(RegNo, Name, DeptID)
DEPARTMENT(DepID, DeptName)

1. Retrieve student names along with their department names.

π Name, DeptName
(STUDENT $\bowtie$ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT)

2. Retrieve students belonging to the CSE department.

π RegNo, Name
(σ DeptName = 'CSE'
(STUDENT $\bowtie$ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT))

3. Retrieve students whose department is CSE and Registration Number is greater than 101.

π RegNo, Name
(σ DeptName = 'CSE' $\wedge$ RegNo > 101
(STUDENT $\bowtie$ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT))

4. Retrieve students whose department is either CSE or ECE.

π RegNo, Name
(σ DeptName = 'CSE' $\vee$ DeptName = 'ECE'
(STUDENT $\bowtie$ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT))

5. Retrieve all department names.

π DeptName
(DEPARTMENT)

3. Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

```
1  -- STUDENT MANAGEMENT SYSTEM
2
3  -- Create Department Table
4  CREATE TABLE DEPARTMENT
5  (
6      DeptID INT PRIMARY KEY,
7      DeptName VARCHAR(20)
8  );
9
10 -- Create Student Table
11 CREATE TABLE STUDENT
12 (
13     RegNo INT PRIMARY KEY,
14     Name VARCHAR(30),
15     Gender CHAR(1),
16     DOB DATE,
17     DeptID INT,
18     FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
19 );
20
21 -- Insert Records into Department Table
22 INSERT INTO DEPARTMENT VALUES
23 (1, 'CSE'),
24 (2, 'ECE'),
25 (3, 'MECH');
26
27 -- Insert Records into Student Table
28 INSERT INTO STUDENT VALUES
29 (101, 'Arun', 'M', '2004-05-10', 1),
30 (102, 'Priya', 'F', '2005-01-20', 2),
31 (103, 'Rahul', 'M', '2004-11-15', 1),
32 (104, 'Sneha', 'F', '2005-03-18', 3);
```

Query 1: Display All Student Records

```
35  SELECT * FROM STUDENT;
```

OUTPUT:

RegNo	Name	Gender	DOB	DeptID
101	Arun	M	2004-05-10	1
102	Priya	F	2005-01-20	2
103	Rahul	M	2004-11-15	1
104	Sneha	F	2005-03-18	3

Query 2: Display Student Name with Department Name

```

38  SELECT S.RegNo, S.Name, D.DeptName
39  FROM STUDENT S
40  INNER JOIN DEPARTMENT D
41  ON S.DeptID = D.DeptID;

```

OUTPUT:

RegNo	Name	DeptName
101	Arun	CSE
103	Rahul	CSE
102	Priya	ECE
104	Sneha	MECH

Query 3: Display Students Belonging to the CSE Department

```

44  SELECT Name
45  FROM STUDENT
46  WHERE DeptID =
47  (
48      SELECT DeptID
49      FROM DEPARTMENT
50      WHERE DeptName = 'CSE'
51  );

```

OUTPUT:

Name
Arun
Rahul

Query 4: Display Students Belonging to the ECE Department

```

54  SELECT Name
55  FROM STUDENT
56  WHERE DeptID =
57  (
58      SELECT DeptID
59      FROM DEPARTMENT
60      WHERE DeptName = 'ECE'
61  );

```

OUTPUT:

Name
Priya

Query 5: Update the Department of Student with RegNo = 101

```
64 UPDATE STUDENT
65 SET DeptID = 2
66 WHERE RegNo = 101;
```

OUTPUT:

DeptName	TotalStudents
ECE	2
MECH	1

4. Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

```
1  -- Create Department Table
2  CREATE TABLE DEPARTMENT
3  (
4      DeptID INT PRIMARY KEY,
5      DeptName VARCHAR(20)
6  );
7
8  -- Create Student Table
9  CREATE TABLE STUDENT
10 (
11     RegNo INT PRIMARY KEY,
12     Name VARCHAR(30),
13     Gender CHAR(1),
14     DOB DATE,
15     DeptID INT,
16     FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
17 );
18
19 -- Insert Sample Data
20 INSERT INTO DEPARTMENT VALUES
21 (1,'CSE'),
22 (2,'ECE'),
23 (3,'MECH');
24
25 INSERT INTO STUDENT VALUES
26 (101,'Arun','M','2004-05-10',1),
27 (102,'Priya','F','2005-01-20',2),
28 (103,'Rahul','M','2004-11-15',1),
29 (104,'Sneha','F','2005-03-18',3);
30
31 -- Nested Subquery
32 SELECT Name
33 FROM STUDENT
34 WHERE DeptID =
35 (
36     SELECT DeptID
37     FROM DEPARTMENT
38     WHERE DeptName = 'CSE'
39 );
```

OUTPUT:

```
+-----+
| Name  |
+-----+
| Arun  |
| Rahul |
+-----+
```

5. Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency.

```
1  CREATE TABLE DEPARTMENT(
2  DeptID INT PRIMARY KEY,
3  DeptName VARCHAR(20));
4
5  CREATE TABLE STUDENT(
6  RegNo INT PRIMARY KEY,
7  Name VARCHAR(30),
8  Gender CHAR(1),
9  DOB DATE,
10 DeptID INT,
11 FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID));
12
13 INSERT INTO DEPARTMENT VALUES
14 (1, 'CSE'),
15 (2, 'ECE'),
16 (3, 'MECH');
17
18 INSERT INTO STUDENT VALUES
19 (101, 'Arun', 'M', '2004-05-10', 1),
20 (102, 'Priya', 'F', '2005-01-20', 2),
21 (103, 'Rahul', 'M', '2004-11-15', 1),
22 (104, 'Sneha', 'F', '2005-03-18', 3);
23
24 SELECT S.Name
25 FROM STUDENT S
26 INNER JOIN DEPARTMENT D
27 ON S.DeptID=D.DeptID
28 WHERE D.DeptName='CSE';
29
30 SELECT Name
31 FROM STUDENT
32 WHERE DeptID=
32 WHERE DeptID=
33 (SELECT DeptID
34 FROM DEPARTMENT
35 WHERE DeptName='CSE');
```

OUTPUT:

```
+-----+
| Name  |
+-----+
| Arun  |
| Rahul |
+-----+
+-----+
| Name  |
+-----+
| Arun  |
| Rahul |
+-----+
```

6. Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

```
1  CREATE TABLE DEPARTMENT(
2  DeptID INT PRIMARY KEY,
3  DeptName VARCHAR(20));
4
5  CREATE TABLE STUDENT(
6  RegNo INT PRIMARY KEY,
7  Name VARCHAR(30),
8  Gender CHAR(1),
9  DOB DATE,
10 DeptID INT,
11 FOREIGN KEY(DeptID) REFERENCES DEPARTMENT(DeptID));
12
13 INSERT INTO DEPARTMENT VALUES
14 (1, 'CSE'),
15 (2, 'ECE'),
16 (3, 'MECH');
17
18 INSERT INTO STUDENT VALUES
19 (101, 'Arun', 'M', '2004-05-10', 1),
20 (102, 'Priya', 'F', '2005-01-20', 2),
21 (103, 'Rahul', 'M', '2004-11-15', 1),
22 (104, 'Sneha', 'F', '2005-03-18', 3);
23
24 CREATE VIEW Student_View AS
25 SELECT S.RegNo, S.Name, D.DeptName
26 FROM STUDENT S
27 JOIN DEPARTMENT D
28 ON S.DeptID=D.DeptID;
29
30 SELECT * FROM Student_View;
31
32 CREATE VIEW CSE_Students AS
33 SELECT RegNo, Name FROM STUDENT WHERE DeptID=1;
34
35 SELECT * FROM CSE_Students;
```

OUTPUT:

RegNo	Name	DeptName
101	Arun	CSE
103	Rahul	CSE
102	Priya	ECE
104	Sneha	MECH

RegNo	Name
101	Arun
103	Rahul

7. Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

STUDENT(RegNo, Name, Gender, DOB, DeptID)
DEPARTMENT(DepID, DeptName)

-- Relational Algebra Expressions

1. Display Student Name and Department Name

π Name, DeptName

(STUDENT $\bowtie$ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT)

2. Display Students Belonging to the CSE Department

π Name

(σ DeptName='CSE'

(STUDENT $\bowtie$ STUDENT.DeptID=DEPARTMENT.DeptID DEPARTMENT))

3. Display Registration Number and Name of Students with DeptID = 1

π RegNo, Name

(σ DeptID=1(STUDENT))

8. Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.

```
1  -- Create Department Table
2  CREATE TABLE DEPARTMENT(
3      DeptID INT PRIMARY KEY,
4      DeptName VARCHAR(20)
5  );
6
7  -- Create Student Table
8  CREATE TABLE STUDENT(
9      RegNo INT PRIMARY KEY,
10     Name VARCHAR(30),
11     Gender CHAR(1),
12     DOB DATE,
13     DeptID INT,
14     FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
15 );
16
17 -- Insert Data
18 INSERT INTO DEPARTMENT VALUES
19 (1, 'CSE'),
20 (2, 'ECE'),
21 (3, 'MECH');
22
23 INSERT INTO STUDENT VALUES
24 (101, 'Arun', 'M', '2004-05-10', 1),
25 (102, 'Priya', 'F', '2005-01-20', 2),
26 (103, 'Rahul', 'M', '2004-11-15', 1),
27 (104, 'Sneha', 'F', '2005-03-18', 3);
28
```

```

29  -- Inefficient Query
30  SELECT Name
31  FROM STUDENT
32  WHERE DeptID=(SELECT DeptID
33  | | | | | | | FROM DEPARTMENT
34  | | | | | | | WHERE DeptName='CSE');
35
36  -- Optimized Query
37  SELECT S.Name
38  FROM STUDENT S
39  JOIN DEPARTMENT D
40  ON S.DeptID=D.DeptID
41  WHERE D.DeptName='CSE';
42
43  -- Create Index
44  CREATE INDEX idx_deptid
45  ON STUDENT(DeptID);
46
47  -- Search Using Index
48  SELECT * FROM STUDENT
49  WHERE DeptID=1;
50
51  -- Create View
52  CREATE VIEW Student_View AS
53  SELECT S.RegNo,S.Name,D.DeptName
54  FROM STUDENT S
55  JOIN DEPARTMENT D
56  ON S.DeptID=D.DeptID;
57
58  -- Display View
59  SELECT * FROM Student_View;

```

OUTPUT:

```
+-----+
| Arun  |
| Rahul |
+-----+
```

```
+-----+
```

```
+-----+
| Name  |
+-----+
```

```
+-----+
```

```
| Arun  |
| Rahul |
+-----+
```

```
+-----+
```

```
+-----+-----+-----+-----+-----+
| RegNo | Name  | Gender | DOB          | DeptID |
+-----+-----+-----+-----+-----+
```

```
+-----+-----+-----+-----+-----+
```

```
| 101   | Arun  | M      | 2004-05-10   | 1      |
+-----+-----+-----+-----+-----+
```

```
| 103   | Rahul | M      | 2004-11-15   | 1      |
+-----+-----+-----+-----+-----+
```

```
+-----+-----+-----+-----+-----+
```

```
+-----+-----+-----+
```

```
| RegNo | Name  | DeptName |
+-----+-----+-----+
```

```
+-----+-----+-----+
```

```
| 101   | Arun  | CSE      |
+-----+-----+-----+
```

```
| 103   | Rahul | CSE      |
+-----+-----+-----+
```

```
| 102   | Priya | ECE      |
+-----+-----+-----+
```

```
| 104   | Sneha | MECH     |
+-----+-----+-----+
```

```
+-----+-----+-----+
```

9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

```

1  -- Create Department Table
2  CREATE TABLE DEPARTMENT(
3  DeptID INT PRIMARY KEY,
4  DeptName VARCHAR(20));
5
6  -- Create Student Table
7  CREATE TABLE STUDENT(
8  RegNo INT PRIMARY KEY,
9  Name VARCHAR(30),
10 Gender CHAR(1),
11 DOB DATE,
12 DeptID INT,
13 FOREIGN KEY(DeptID) REFERENCES DEPARTMENT(DeptID));
14
15 -- Insert Data
16 INSERT INTO DEPARTMENT VALUES
17 (1,'CSE'),
18 (2,'ECE'),
19 (3,'MECH');
20
21 INSERT INTO STUDENT VALUES
22 (101,'Arun','M','2004-05-10',1),
23 (102,'Priya','F','2005-01-20',2),
24 (103,'Rahul','M','2004-11-15',1),
25 (104,'Sneha','F','2005-03-18',3);
26
27 -- Grouping and Aggregation
28 SELECT DeptID,COUNT(*) AS TotalStudents
29 FROM STUDENT
30 GROUP BY DeptID;
31
32 -- Nested Query with Aggregation
33 SELECT DeptID,COUNT(*) AS TotalStudents
34 FROM STUDENT
35 GROUP BY DeptID
36 HAVING COUNT(*)=
37 (
38 SELECT MAX(StudentCount)
39 FROM
40 (
41 SELECT COUNT(*) AS StudentCount
42 FROM STUDENT
43 GROUP BY DeptID
44 ) AS T
45 );

```

OUTPUT:

```

+-----+-----+
| DeptID | TotalStudents |
+-----+-----+
|      1 |             2 |
|      2 |             1 |
|      3 |             1 |
+-----+-----+
+-----+-----+
| DeptID | TotalStudents |
+-----+-----+
|      1 |             2 |
+-----+-----+

```


10. Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.

```
1  -- Create Department Table
2  CREATE TABLE DEPARTMENT(
3  DeptID INT PRIMARY KEY,
4  DeptName VARCHAR(20));
5
6  -- Create Student Table
7  CREATE TABLE STUDENT(
8  RegNo INT PRIMARY KEY,
9  Name VARCHAR(30),
10 Gender CHAR(1),
11 DOB DATE,
12 DeptID INT,
13 FOREIGN KEY(DeptID) REFERENCES DEPARTMENT(DeptID));
14
15 -- Insert Data
16 INSERT INTO DEPARTMENT VALUES
17 (1, 'CSE'),
18 (2, 'ECE'),
19 (3, 'MECH');
20
21 INSERT INTO STUDENT VALUES
22 (101, 'Arun', 'M', '2004-05-10', 1),
23 (102, 'Priya', 'F', '2005-01-20', 2),
24 (103, 'Rahul', 'M', '2004-11-15', 1),
25 (104, 'Sneha', 'F', '2005-03-18', 3);
26
```

```

27  -- JOIN
28  SELECT S.RegNo,S.Name,D.DeptName
29  FROM STUDENT S
30  JOIN DEPARTMENT D
31  ON S.DeptID=D.DeptID;
32
33  -- SUBQUERY
34  SELECT Name
35  FROM STUDENT
36  WHERE DeptID=
37  (SELECT DeptID
38  FROM DEPARTMENT
39  WHERE DeptName='CSE');
40
41  -- VIEW
42  CREATE VIEW student_View AS
43  SELECT S.RegNo,S.Name,D.DeptName
44  FROM STUDENT S
45  JOIN DEPARTMENT D
46  ON S.DeptID=D.DeptID;
47
48  SELECT * FROM student_View;

```

OUTPUT:

```

+-----+-----+-----+
| RegNo | Name  | DeptName |
+-----+-----+-----+
| 101   | Arun  | CSE      |
| 103   | Rahul | CSE      |
| 102   | Priya | ECE      |
| 104   | Sneha | MECH     |
+-----+-----+-----+

+-----+
| Name  |
+-----+
| Arun  |
| Rahul |
+-----+

+-----+-----+-----+
| RegNo | Name  | DeptName |
+-----+-----+-----+
| 101   | Arun  | CSE      |
| 103   | Rahul | CSE      |
| 102   | Priya | ECE      |
| 104   | Sneha | MECH     |
+-----+-----+-----+

```